Chapter 9: Virtual Memory

Paging

Paging divides a process's logical address space into fixed-size **pages** and physical memory into **frames** of the same size. Any page can be placed in any free frame, so a process does not need a contiguous region of physical memory and there is **no external fragmentation**. The cost is **internal fragmentation**: on average half of the last page of each process is wasted.

Address translation

A logical address is split into a page number p and an offset d. The page number indexes the page table, which gives the frame number f; the physical address is f followed by the same offset. With 4 KiB pages the offset is 12 bits, because $2^{12} = 4096$. A 32-bit address with 4 KiB pages therefore has a 20-bit page number, so a flat page table has 2^{20} (about one million) entries.

Why pages are a power of two

Because the page size is a power of two, splitting an address needs no division: the hardware simply takes the high-order bits as the page number and the low-order bits as the offset.

Page Tables and the TLB

Page table entries

Besides the frame number, each page table entry holds control bits: a **valid** bit (the page is in memory), a **dirty** bit (the page was written and must be saved before eviction), a **referenced** bit (set on any access, used by replacement algorithms) and **protection** bits (read, write, execute).

Multi-level page tables

A flat table for a 64-bit address space would be enormous, so page tables are hierarchical. x86-64 uses four levels for 48-bit virtual addresses: each level is indexed by 9 bits of the address and the last 12 bits are the offset ($4 \times 9 + 12 = 48$). Tables for unused parts of the address space are never allocated.

Translation lookaside buffer

Walking the page table on every access would multiply memory traffic. The **TLB** is a small, fast associative cache of recent translations. With TLB hit ratio h, TLB lookup time t_{tlb} and memory access time t_{mem}, the effective access time is

$$EAT = h \times (t_{tlb} + t_{mem}) + (1 - h) \times (t_{tlb} + 2 \times t_{mem})$$

For example, with a 98% hit ratio, a 20 ns TLB and 100 ns memory: $EAT = 0.98 \times 120 + 0.02 \times 220 = 122$ ns, only 22% slower than a single memory access.

Demand Paging and Page Faults

With **demand paging** a page is loaded only when it is first used. An access to a page whose valid bit is clear causes a **page fault**, which the operating system handles as follows:

1. The hardware traps to the kernel, saving the process state.
2. The kernel checks that the reference is legal; an illegal one terminates the process.
3. It finds a free frame, running the page-replacement algorithm if none is free.
4. It schedules a disk read of the page into the frame; the process waits meanwhile.
5. When the read completes it updates the page table and sets the valid bit.
6. It restarts the instruction that caused the fault.

Cost of page faults

With page-fault rate p, a 200 ns memory access and an 8 ms fault service time, EAT = (1 – p) × 200 + p × 8,000,000 ns. Keeping the slowdown under 10% (EAT < 220 ns) requires p < 0.0000025, fewer than one fault in about 400,000 accesses. Disk is so slow that even rare faults dominate performance.

Copy-on-write

After fork(), parent and child initially share all pages, marked read-only. A page is copied only when one of them writes to it, which makes fork() followed by exec() cheap.

Page Replacement

When no frame is free, the kernel must choose a victim page. A dirty victim is written back first.

- **FIFO** evicts the page that has been in memory longest. It is simple but can evict heavily used pages.
- **Optimal (OPT, Belady's MIN)** evicts the page that will not be used for the longest time. It needs future knowledge, so it serves only as a benchmark.
- **LRU** evicts the page unused for the longest time, approximating OPT by looking backwards. Exact LRU needs timestamps or a stack on every access, which is too costly in hardware.
- **Clock (second chance)** approximates LRU with the referenced bit, as below.

Belady's anomaly

With FIFO, adding frames can increase the number of faults. For the reference string 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5, FIFO causes 9 faults with three frames but 10 faults with four. LRU and OPT are **stack algorithms** and never show the anomaly.

Clock algorithm

```
def clock_evict(frames, hand):
    while True:
        page = frames[hand]
        if page.referenced:
            page.referenced = False      # second chance
            hand = (hand + 1) % len(frames)
        else:
            return hand                  # victim found
```

Thrashing and the Working Set

A process **thrashes** when it spends more time paging than executing. It happens when the processes in memory together need more frames than exist: each fault evicts a page that is needed again soon. CPU utilisation drops, and a naive scheduler reacts by admitting more processes, which makes it worse.

Working-set model

The **working set** of a process is the set of pages it referenced in the most recent Δ memory references (the working-set window). If the sum of all working-set sizes exceeds the number of frames, thrashing is imminent and the operating system should **suspend** a process, freeing its frames, rather than let every process thrash. Choosing Δ is a trade-off: too small misses the current locality, too large spans several localities.

Page-fault frequency

A simpler control watches each process's page-fault rate directly. With an upper and a lower bound, a process faulting above the upper bound is given more frames and one faulting below the lower bound loses frames. If no frames are free when a process needs more, a process is swapped out.

Summary

Virtual memory lets programs be larger than physical memory and lets processes share code, at the price of translation hardware (the TLB) and careful replacement policies.